# Experiment 6

**Aim:** To study and implement the Agile Software Project lifecycle using Jira with DevOps Pipeline Integration.

**LO mapped:** LO1, LO3

## Theory

### Agile and Scrum

Agile is an iterative approach to software development that emphasises incremental delivery, continuous feedback, and close collaboration between cross-functional teams. Scrum is the most widely used Agile framework and organises work into fixed-length iterations called Sprints, backed by a Product Backlog (all planned work) and a Sprint Backlog (work committed for the current Sprint).

### Jira for Agile Project Management

Jira is a project management and issue-tracking tool built around Agile workflows. It lets a team break a project down into Epics (large bodies of related work), and further into Stories/Tasks that can be estimated, assigned, and tracked on a Backlog or Board. Team members can be added as collaborators so that work items can be assigned and progress is visible to everyone on the project.

### DevOps and CI/CD

DevOps extends Agile by integrating development and operations, using automation to build, test, and deploy code continuously. Continuous Integration (CI) automatically builds and tests code on every push, while Continuous Delivery/Deployment (CD) automates release of validated changes. GitHub Actions is a CI/CD service built into GitHub that runs configurable workflows (e.g. running a test suite) whenever code is pushed or a pull request is opened.

### Jira–GitHub Integration

Atlassian's GitHub for Jira app links a GitHub organisation/repository to a Jira project. Once connected, any commit, branch, or pull request whose message or name contains a Jira issue key (e.g. SCRUM-11) is automatically linked to that issue and appears in its Development panel. If the linked repository also runs GitHub Actions, the corresponding build/pipeline status is surfaced in the same panel — giving full traceability from a Jira work item to the exact code change and its CI result, without leaving Jira.

## Procedure

### 1. Project Setup and Team Collaboration

A new Scrum project ("My Software Team") was created in Jira for the chosen application — a Campus Event Management System, a platform for students to discover, register for, and manage campus events. Team collaborators were added to the project so that work items could be assigned and tracked collectively.

### 2. Epic Planning

The work was broken down into the following Epics, reviewed and reorganised to cleanly map to the modules of the Campus Event Management System:

- User Authentication — registration, login/logout, password validation, profile management
- Event Management — creating and listing events
- Event Discovery — browsing/viewing upcoming events
- Event Registration — registering for an event
- Admin & Reporting — administrative and reporting functionality
- Testing & Deployment — automated testing and the CI/CD pipeline

### 3. Sprint Planning

Sprint 1 (4 Sep – 18 Sep) was planned around a minimum viable product goal of "Auth + Event create/list + Registration + CI", and the following six issues were moved from the Backlog into the Sprint:

- SCRUM-11: User registration
- SCRUM-12: User login/logout
- SCRUM-15: Create event
- SCRUM-19: View upcoming events
- SCRUM-33: CI/CD
- SCRUM-23: Register for event

The screenshot below shows the finalised Sprint Backlog, with the Epics panel on the left and Sprint 1's committed work items on the right (remaining items stay in the general Backlog).

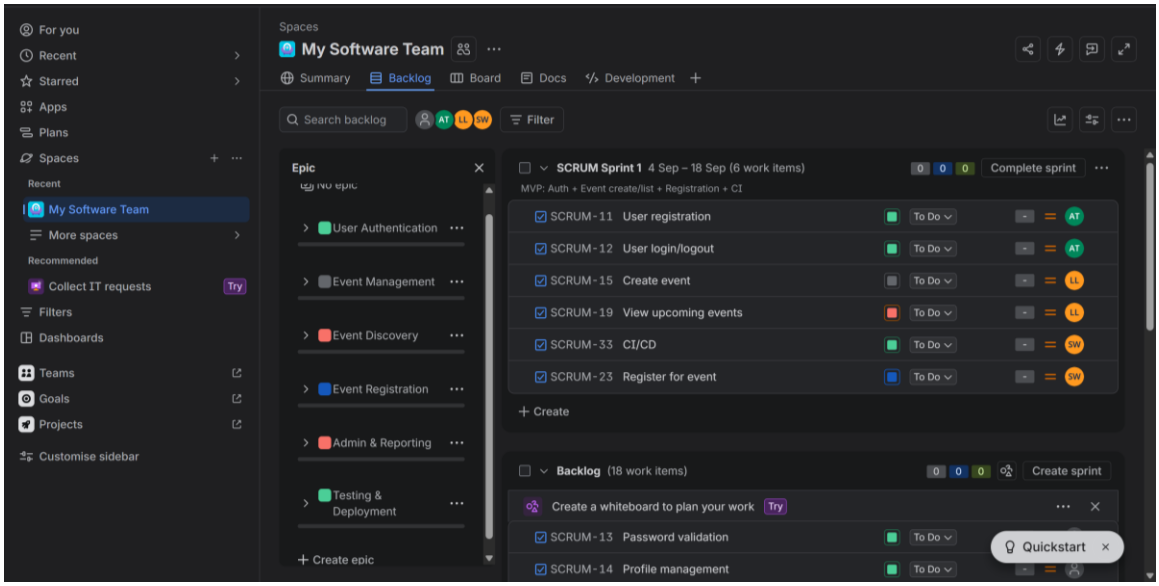



*Fig 1. Jira Backlog view — Epics (left) and Sprint 1 work items (right)*

## 4. Connecting GitHub to Jira

The GitHub for Jira app was configured from the Jira project settings, connecting the GitHub organisation/account (akshaytolwani123) to Jira with full access. This enables Jira to automatically detect and link commits, branches, and builds from the connected repository.

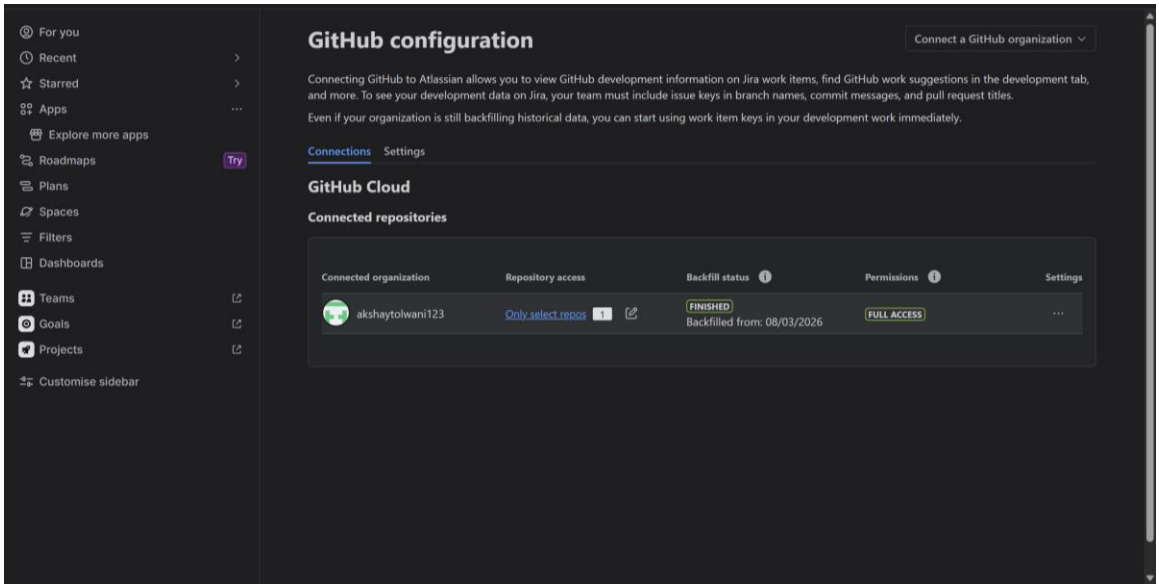



*Fig 2. GitHub configuration screen showing the connected GitHub organisation with full access and completed backfill*

## 5. Writing and Pushing Code

A repository, asd-jira-campus-event-management, was created and an MVP Flask API was implemented covering authentication, event creation/listing, and registration, followed by a pytest suite for Sprint 1's acceptance criteria and a GitHub Actions workflow that runs CI on every push and pull request to main. Each commit message referenced the relevant Jira issue keys (a "Smart Commit" style), which is what triggers the automatic Jira–GitHub link.

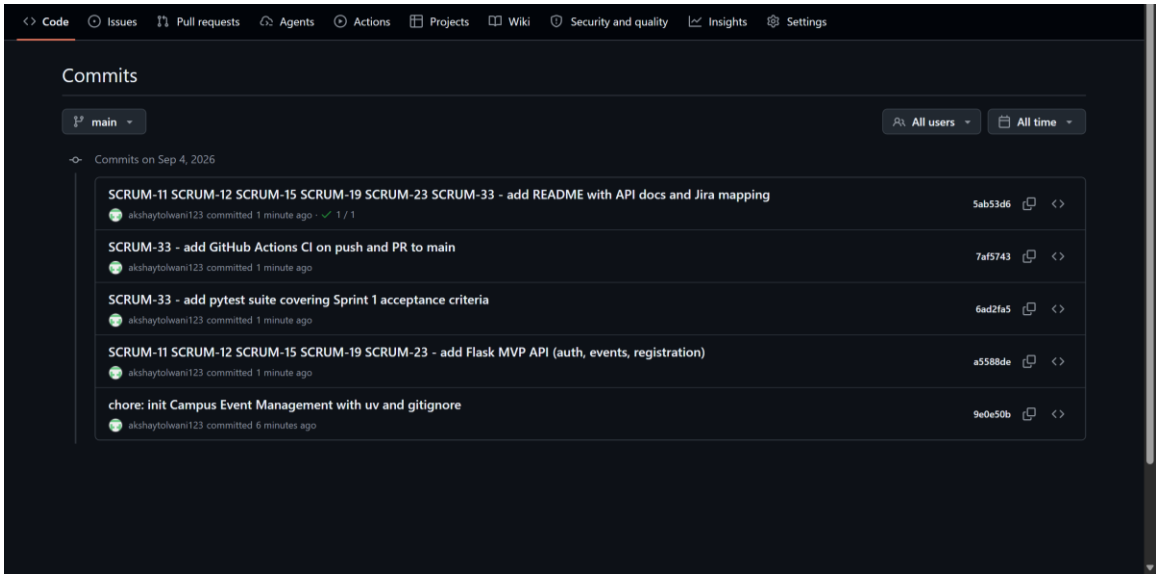



*Fig 3. GitHub commit history for the repository, with commit messages referencing SCRUM issue keys*

The repository itself confirms the structure of the MVP: a Flask app (app.py), a tests folder with the pytest suite, a .github/workflows folder holding the CI pipeline definition, and a README documenting the API and its mapping to Jira issues.

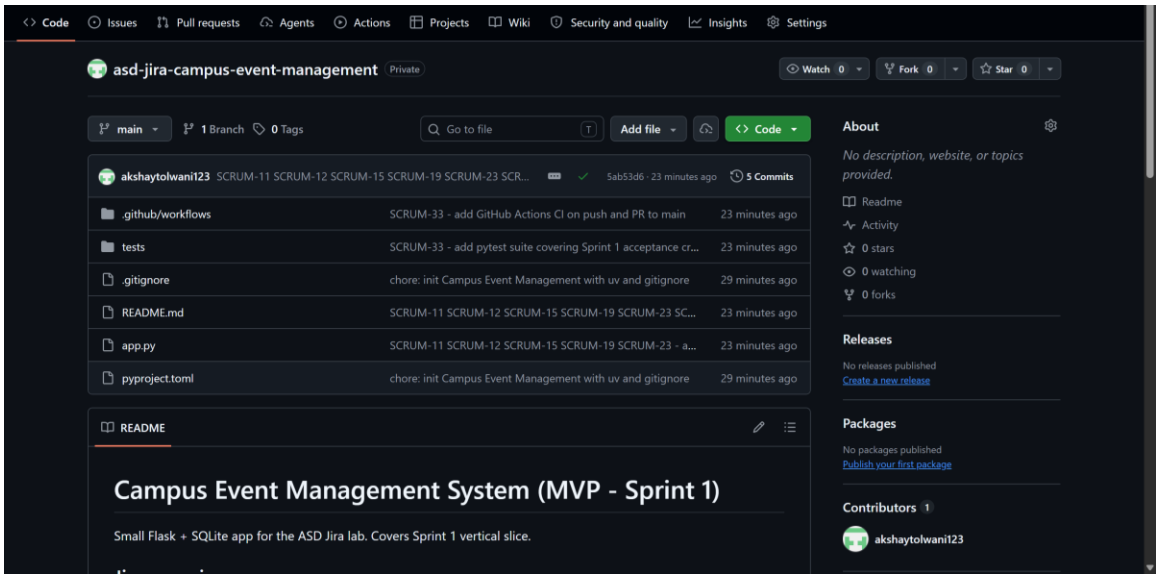



*Fig 4. GitHub repository view — app.py, tests/, .github/workflows/, and README.md for the Campus Event Management System MVP*

## 6. Verifying the Jira–GitHub–CI/CD Integration

Opening issue SCRUM-11 (User registration) in Jira confirms it is correctly nested under the Epic SCRUM-5 (User Authentication) and assigned to Sprint 1

*Fig 5. SCRUM-11 issue detail — Parent Epic and Sprint fields confirm correct backlog structure*

The issue's Development panel automatically lists every linked commit from the connected repository, confirming that pushing code with the issue key in the commit message links the change back to Jira without any manual step:

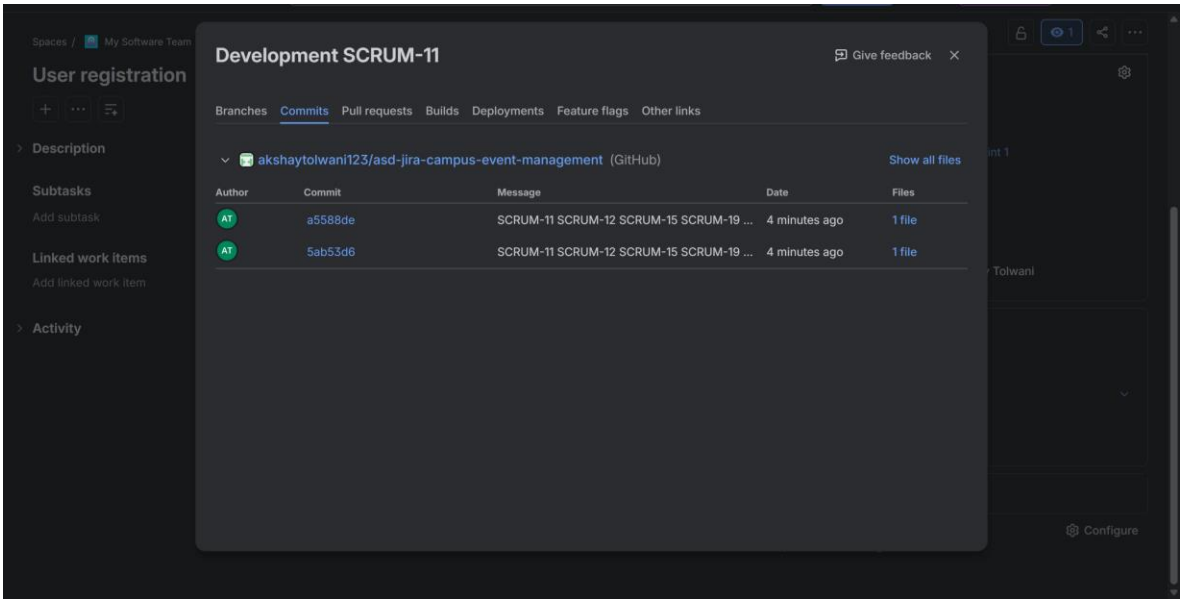



*Fig 6. Development panel on SCRUM-11 showing linked GitHub commits*

The same Development panel also surfaces the GitHub Actions CI pipeline runs for those commits, both of which completed successfully — demonstrating the full DevOps loop from code push to automated build/test result, visible directly inside the Jira issue:

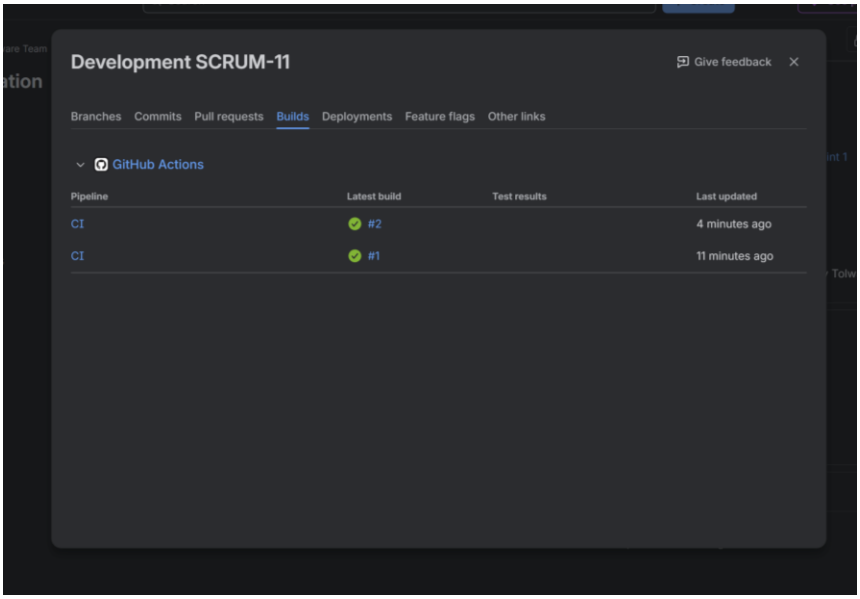



*Fig 7. Development panel — Builds tab showing successful GitHub Actions CI runs (#1 and #2) linked to SCRUM-11*

The GitHub Actions runs referenced in that Builds tab can also be inspected directly on GitHub. The workflow's run history shows both CI runs completing successfully, each triggered automatically by a push to main:

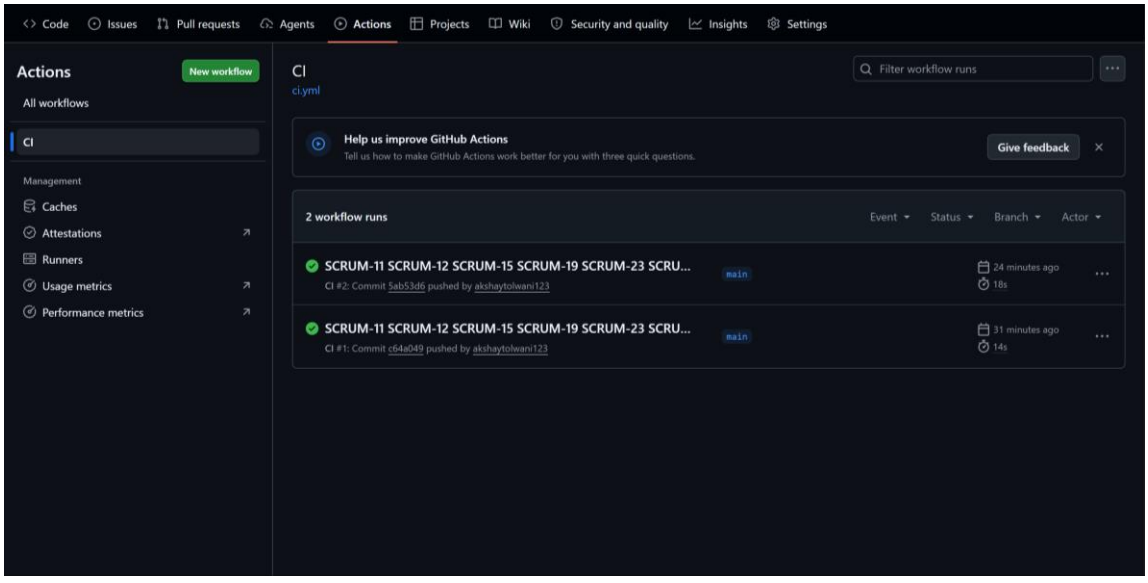



*Fig 8. GitHub Actions — CI workflow run history showing both runs passing, triggered by pushes to main*

Drilling into a run shows the individual pipeline steps — checking out the code, setting up uv and Python, installing dependencies, and running the pytest suite — all completing successfully:

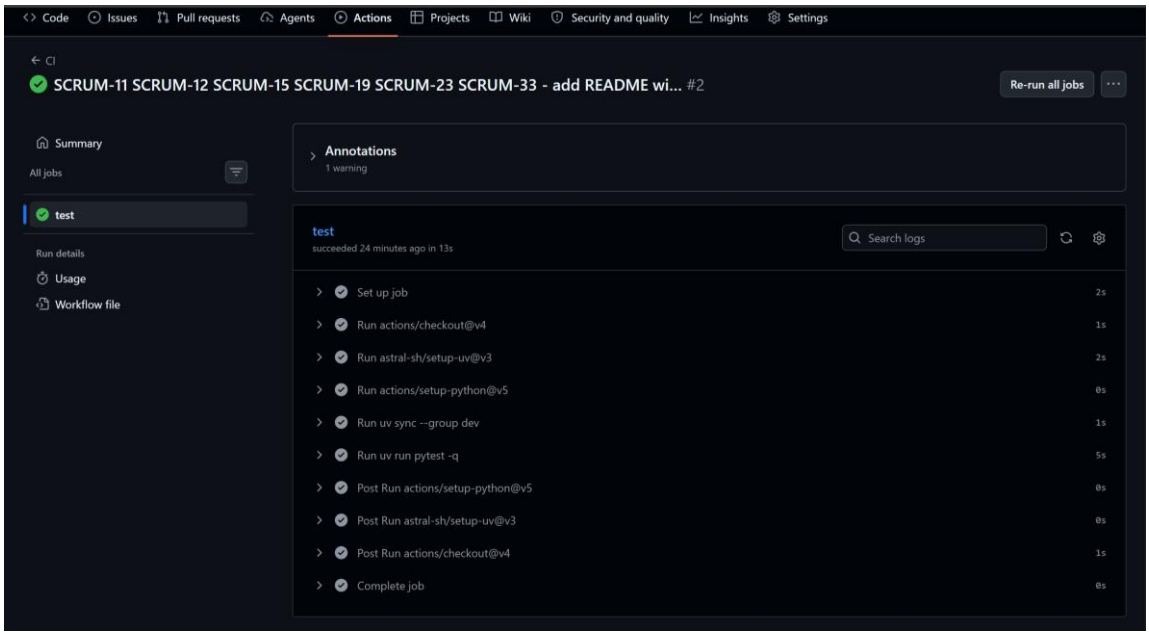



*Fig 9. GitHub Actions — detailed job steps for a successful CI run (checkout, environment setup, dependency install, pytest)*

## 7. Traceability Summary

The table below summarises the mapping between Sprint 1's Jira issues, the Epics they belong to, and the GitHub activity linked to each of them.

| Jira Issue | Summary | Epic | Linked GitHub Activity |
|---|---|---|---|
| SCRUM-11 | User registration | User Authentication | Commit a5588de, 5ab53d6 |
| SCRUM-12 | User login/logout | User Authentication | Commit a5588de, 5ab53d6 |
| SCRUM-15 | Create event | Event Management | Commit a5588de, 5ab53d6 |
| SCRUM-19 | View upcoming events | Event Discovery | Commit a5588de, 5ab53d6 |
| SCRUM-23 | Register for event | Event Registration | Commit a5588de, 5ab53d6 |

| Jira Issue | Summary | Epic | Linked GitHub Activity |
|---|---|---|---|
| SCRUM-33 | CI/CD | Testing & Deployment | GitHub Actions workflow, pytest suite, CI builds #1 & #2 |

## Conclusion

The Agile Software Project lifecycle was successfully studied and implemented for a Campus Event Management System using Jira. A Scrum project was set up with collaborators, work was organised into six Epics, and Sprint 1 was planned with a clear MVP scope. GitHub was connected to Jira, and code pushed with issue-key-referenced commits was automatically linked to the corresponding Jira issues' Development panel, along with GitHub Actions CI build results. This confirmed end-to-end traceability from Sprint planning through to code commits and automated CI/CD pipeline execution, demonstrating a working integration between Agile project management (Jira) and a DevOps pipeline (GitHub + GitHub Actions).